

Escola Superior Náutica Infante D. Henrique

Unidade Curricular: Inteligência Artificial para Sistemas Autónomos

Relatório de Projeto

"Resolução do Puzzle Jogo dos 15"

Autores:

- Gonçalo Taborda – Nº 14065 | 3º ano – LEIC
- Henrique Domingos – Nº 14064 | 3º ano – LEIC
- Manuel Roque – Nº 14061 | 3º ano – LEIC

Docente: Anna Carolina Finamore Couto

Ano Letivo: 2025 / 2026

Índice:

1. Introdução

2. Descrição do Problema

3. Representação do Estado

4. Verificação de Solvabilidade

    a. Exemplo prático com o resultado observado no nosso projeto

5. Métodos de Procura Implementados
    a) Procura em Profundidade LIFO (DFS)
    b) Procura em Largura FIFO (BFS)
    c) GBFS (Greedy Best-First Search)
    d) A*
    e) IDA* (Iterative Deepening A*)

6. Heurísticas Utilizadas
    a) "Posições das peças fora do lugar"
    b) Distância de Manhattan
    c) Conflito Linear

7. Reinforcement Learning

8. Explicação do ficheiro principal; "main.py"

9. Resultados Obtidos

10. Conclusão

1. Introdução

O Jogo dos 15 é um puzzle deslizante clássico, caracterizado por um espaço de estados muito elevado e por regras de movimento bem definidas, o que torna inviáveis abordagens não informadas para configurações não triviais.

Neste projeto são implementados vários algoritmos de procura, nomeadamente DFS, BFS, GBFS, A* e IDA*, recorrendo a heurísticas como a distância de Manhattan e o conflito linear. Para além destas abordagens clássicas, é também explorada uma solução baseada em Reinforcement Learning, utilizando Q-learning, permitindo comparar diferentes paradigmas de resolução do mesmo problema.

2. Descrição do Problema

O problema abordado neste projeto consiste na resolução do Jogo dos 15, um puzzle clássico constituído por uma matriz 4×4 que contêm 15 peças numeradas e um espaço vazio. O objetivo é alcançar um estado final ordenado, considerando duas configurações possíveis: a configuração clássica, em que o espaço vazio se encontra na última posição da matriz, e uma configuração alternativa, em que o espaço vazio ocupa a primeira posição. Em ambos os casos, as restantes peças devem estar dispostas por ordem crescente.

Este problema é um exemplo típico de procura em espaço de estados, caracterizado por um elevado fator de ramificação e por um espaço de estados muito grande, o que inviabiliza abordagens de força bruta para instâncias não triviais. Assim, a sua resolução exige a aplicação de algoritmos de procuras informadas (GBFS; A* e IDA*), suportados por heurísticas adequadas, bem como a análise de abordagens alternativas como Reinforcement Learning.

3. Representação do Estado

Cada estado do Jogo dos 15 é representado por uma matriz 4×4, onde cada posição contém uma peça numerada de 1 a 15 ou o valor 0, que representa o espaço vazio. Esta representação permite descrever de forma direta qualquer configuração do puzzle e identificar facilmente os movimentos possíveis, que resultam da deslocação do espaço vazio nas quatro direções válidas (cima, baixo, esquerda e direita).

Para apoiar os algoritmos de procuras abordados, é utilizada uma estrutura auxiliar que associa cada peça à sua posição no estado objetivo, possibilitando o cálculo eficiente das heurísticas. A representação adotada é flexível, permitindo alterar tanto o estado inicial como o estado objetivo sem necessidade de modificar a lógica dos algoritmos de procura, pois estas representações encontram-se num ficheiro "principal", denominado por "main.py".

4. Verificação de Solvabilidade

Antes da execução de qualquer algoritmo de procura, é realizada uma verificação de solvabilidade, uma vez que, no Jogo dos 15, nem todas as configurações iniciais podem atingir qualquer estado objetivo. Este passo é essencial para evitar a aplicação de algoritmos de procura a instâncias que são matematicamente impossíveis de resolver.

No caso de uma matriz 4×4, a solvabilidade depende de dois fatores:

- o número de inversões do estado inicial;
- a posição da linha do espaço vazio, contada a partir da base da matriz.

Com base nestes dois valores, é calculada a paridade do estado inicial e comparada com a paridade do estado objetivo selecionado. Apenas quando ambas coincidem é que o puzzle é considerado solúvel.

- 4.a) Exemplo prático com o nosso projeto

No âmbito do nosso projeto, o estado inicial considerado foi uma matriz 4×4 com as peças distribuídas de forma não ordenada. Ao testar este estado relativamente a dois objetivos distintos — o objetivo clássico (com o espaço vazio na última posição) e um objetivo alternativo (com o espaço vazio na primeira posição) — observaram-se comportamentos distintos:

Para o objetivo clássico, foi possível obter soluções válidas (em pouco tempo estimado) utilizando algoritmos como GBFS, A*, IDA* e também através de uma abordagem de Reinforcement Learning, confirmando que o estado inicial pertence à mesma classe de paridade do objetivo.

Por outro lado, ao selecionar o objetivo alternativo, nenhuma das procuras implementadas conseguiu encontrar solução possível. Umas por não apresentarem solvabilidade e outras por demorarem demasiado tempo (procuras não informadas), sendo apresentado ao utilizador a seguinte mensagem de erro: *"Este puzzle não é solúvel para o objetivo selecionado."*. Este resultado não se deve a limitações dos algoritmos ou das heurísticas utilizadas, mas sim ao facto de o estado inicial e o objetivo alternativo pertencerem a classes de paridade diferentes.

Assim, a verificação de solvabilidade permite identificar corretamente estas situações e justifica, de forma teórica e experimental, a inexistência de solução para determinados pares estado–objetivo.

5. Métodos de Procura Implementados

Foram implementados e analisados os seguintes algoritmos de procura:

- 5.1) Procura em Profundidade (DFS – Procura não informada)

A procura em profundidade explora o espaço de estados seguindo um caminho até à profundidade máxima antes de retroceder. Embora utilize pouca memória, não garante soluções ótimas e pode apresentar tempos de execução elevados.

- 5.2) Procura em Largura (BFS - Procura não informada)

A procura em largura garante a obtenção da solução com o menor número de movimentos, mas o seu consumo de memória cresce exponencialmente, tornando-a impraticável para instâncias não triviais do problema.

- 5.3) Greedy Best-First Search (GBFS - Procura informada)

A GBFS utiliza apenas uma heurística para guiar a procura, escolhendo sempre o estado aparentemente mais próximo do objetivo. No entanto, ignora o custo acumulado do caminho, o que faz com que obtenha um elevado nº de movimentos na sua resolução.

- 5.4) A* (Procura não informada)

O algoritmo A* combina o custo acumulado do caminho com uma estimativa heurística do custo restante. Quando utilizado com heurísticas admissíveis, garante a obtenção de soluções ótimas. Contudo, o seu principal inconveniente reside no elevado consumo de memória, resultante da manutenção de uma fronteira extensa de estados.

- 5.5) IDA* (Iterative Deepening A* - Procura não informada)

O IDA* combina a estratégia do A* com a eficiência em memória da procura em profundidade. O algoritmo executa sucessivas pesquisas em profundidade com limites progressivos sobre a função de avaliação f(n), garantindo a veracidade da solução quando utilizado com heurísticas admissíveis.

Apesar de poder reexplorar estados já visitados, o IDA* apresenta um consumo de memória significativamente inferior ao do A*, o que o torna particularmente adequado para problemas com grandes espaços de estados, como o Jogo dos 15.

6. Heurísticas Utilizadas

- 6.1) “Posições das peças fora do lugar”

Esta heurística contabiliza o número de peças que não se encontram na posição correta. Apesar de admissível, revelou-se pouco informativa para este problema.

- 6.2) Distância de Manhattan

A distância de Manhattan calcula a soma das distâncias horizontais e verticais de cada peça até à sua posição final. Trata-se de uma heurística admissível e consistente, amplamente utilizada em puzzles deslizantes.

- 6.3) Conflito Linear

O conflito linear estende a distância de Manhattan ao penalizar situações em que duas peças se encontram na linha ou coluna corretas, mas em ordem inversa. Esta heurística é mais informativa e mantém a admissibilidade.

7. Reinforcement Learning

Para além das procuras clássicas, foi implementada uma abordagem baseada em Reinforcement Learning, recorrendo ao algoritmo de Q-learning. Nesta abordagem, o agente aprende uma política ótima através da interação com o ambiente, recebendo recompensas associadas à melhoria da distância de Manhattan e à obtenção do estado objetivo, em vez de explorar explicitamente o espaço de estados como nas procuras informadas.

Para o objetivo clássico, o agente conseguiu aprender uma política que conduz à solução do puzzle, obtendo um número de movimentos equivalente ao alcançado pelos algoritmos A* e IDA*. No entanto, este resultado é obtido com um custo computacional substancialmente superior, uma vez que o processo de treino requer dezenas de milhares de episódios e um tempo de execução que ainda demora alguns minutos (+ ou – 2min).

Ao contrário do A* e do IDA*, que utilizam diretamente uma função heurística para guiar a pesquisa de forma determinística, o Reinforcement Learning baseia-se num processo iterativo e estocástico de aprendizagem, sendo menos eficiente para problemas determinísticos e totalmente observáveis como o Jogo dos 15. Ainda assim, esta abordagem revelou-se útil para demonstrar a aplicabilidade de técnicas de aprendizagem por reforço em problemas de procura em espaço de estados.

Tal como nas procuras clássicas, o Reinforcement Learning não conseguiu resolver o puzzle quando o objetivo alternativo (peça 0 na primeira posição da matriz) não é compatível com a solvabilidade do estado inicial, reforçando a importância da análise de paridade antes da execução de qualquer algoritmo.

8. Explicação do ficheiro principal; “main.py”

Sendo este o ficheiro “principal” do projeto, o main.py é responsável por dar vida a todo o código desenvolvido, funcionando como o ponto de ligação entre o utilizador e os restantes módulos implementados. É neste ficheiro que toda a lógica de interação é centralizada, permitindo explorar o Jogo dos 15 de forma dinâmica e intuitiva.

Ao iniciar a execução, é apresentado ao utilizador um menu de escolha do objetivo, onde este pode selecionar qual o estado final que pretende atingir. Esta escolha define dinamicamente o estado objetivo global, influenciando diretamente a verificação de solvabilidade, as heurísticas utilizadas e o comportamento dos algoritmos de procura.

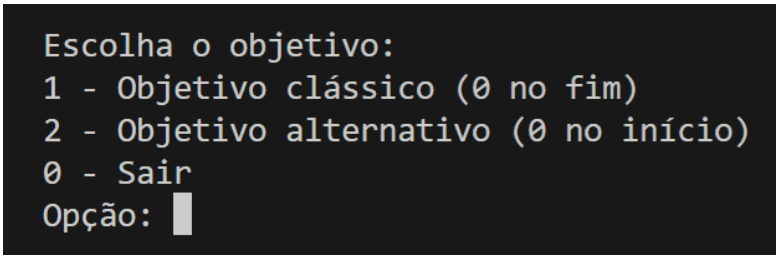


FIG.1 “MENU DE ESCOLHA DO OBJETIVO”

Depois de selecionado o objetivo, o utilizador acede ao menu principal, a partir do qual pode executar uma procura clássica, comparar duas procuras, aplicar Reinforcement Learning, mudar novamente o objetivo ou terminar o programa.

Esta estrutura permite testar diferentes abordagens sem necessidade de reiniciar a aplicação.

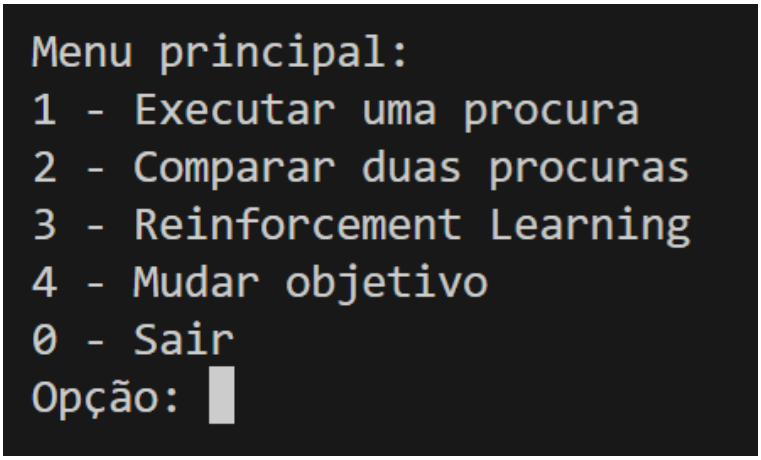


FIG.2 “MENU PRINCIPAL”

Quando o utilizador opta por executar uma procura clássica, é apresentado o menu de procuras clássicas, onde estão disponíveis os algoritmos DFS, BFS, GBFS, A* e IDA*. Antes da execução de qualquer algoritmo, é feita automaticamente a verificação da solvabilidade do puzzle para o objetivo escolhido, evitando execuções desnecessárias sobre estados impossíveis.

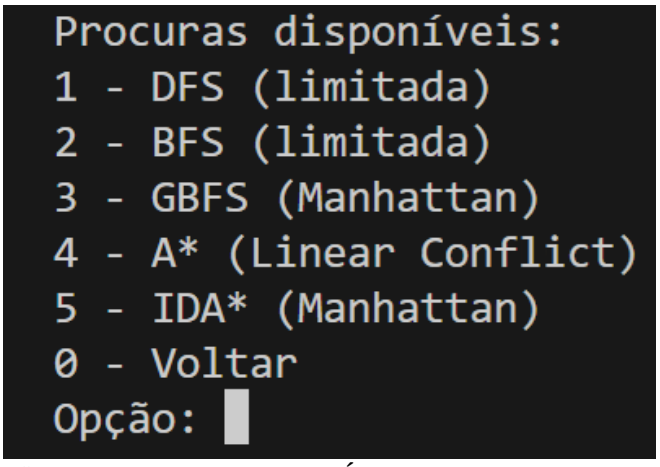


FIG.3 "MENU DE PROCURAS CLÁSSICAS (IMPLEMENTADAS)"

Após a execução de uma procura, os resultados são apresentados passo a passo, mostrando os movimentos realizados e o estado da matriz em cada etapa. De seguida, o utilizador é encaminhado para o menu após execução, onde pode decidir se pretende regressar ao menu principal, alterar o objetivo ou sair do programa.

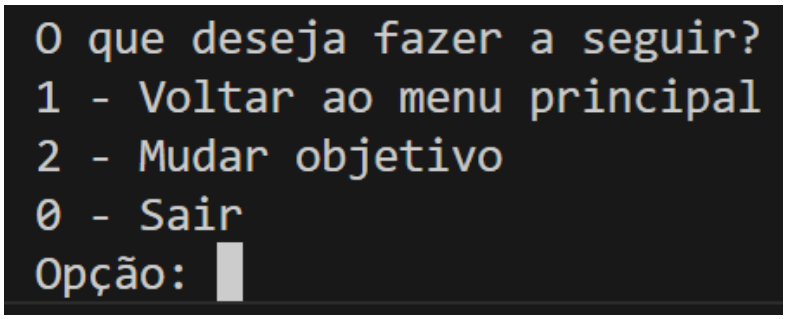


FIG.4 "MENU APÓS EXECUÇÃO"

Este fluxo garante uma interação consistente e controlada, permitindo analisar facilmente o comportamento dos diferentes algoritmos e comparar os seus resultados para distintos estados objetivos, mantendo sempre o foco na clareza e usabilidade da aplicação.

9. Resultados Obtidos

Para o objetivo clássico, os algoritmos A* e IDA* encontraram soluções ótimas com 29 movimentos, o que era esperado, dado o uso de heurísticas admissíveis. Ambos produzem soluções com o mesmo custo, diferenciando-se sobretudo no consumo de memória: o A* utiliza mais memória ao manter uma fronteira extensa, enquanto o IDA* reduz significativamente esse consumo através de pesquisa em profundidade iterativa.

As procuras não informadas mostraram-se pouco adequadas ao problema. O Reinforcement Learning conseguiu também atingir o objetivo clássico com o mesmo número de movimentos, mas exigiu um tempo de execução superior devido à fase de treino. Para o objetivo alternativo, o estado inicial revelou-se não solúvel, sendo corretamente identificado pelo teste de solvabilidade.

10. Conclusão

Ao longo deste trabalho foram analisadas diferentes abordagens para a resolução do Jogo dos 15, incluindo procuras não informadas, procuras informadas e uma abordagem alternativa baseada em aprendizagem por reforço. Os resultados obtidos evidenciam a relevância do uso de heurísticas adequadas em problemas com um espaço de estados vasto e um elevado fator de ramificação.

As procuras informadas, em particular o A* e o IDA*, demonstraram um desempenho significativamente superior às restantes abordagens, permitindo obter soluções ótimas de forma eficiente para o objetivo clássico, em que a peça 0 se encontra na última posição da matriz. Entre estas duas, o A* destaca-se pela rapidez, enquanto o IDA* apresenta uma vantagem clara em termos de consumo de memória, mantendo a qualidade da solução. Assim, o IDA* revela-se especialmente adequado em cenários onde existem restrições de memória.

Por outro lado, as procuras não informadas, como DFS e BFS, mostraram-se pouco eficazes para este problema, sendo fortemente limitadas pelo crescimento exponencial do espaço de estados.